Markdown — die Basis

Was jeder Markdown-Editor versteht — Grundlage für alle LiaScript-Dokumente.

Live-Editor

YAML-Header

```
<!--
author:   Maria Muster
version:  1.0.0
language: de
narrator: Deutsch Female
mode:     Presentation
edit:     true
tags:     OER, LiaScript, Fach
comment:  Interaktiver Kurs
import:   ./template.md
-->
```

Steht ganz oben im Dokument. Definiert Version, Sprache, Startmodus, Bearbeitung, Tags und Beschreibung.

`import:` mit lokalem Pfad oder URL für jeden Baustein wiederholen.

Version & gespeicherte Eingaben

```
<!-- version: 1.0.0 -->
<!-- version: 0.1.0 -->
<!-- version: 0.0.1 -->
```

1.0.0: Release; Quiz- und Codezustände werden im Browser persistent gespeichert.

0.1.0: Entwicklungsstand; bei jedem Laden neu parsen, Eingaben nicht persistent speichern.

0.0.1: früher Debug-/Teststand; gleiches Speicherverhalten wie jede Hauptversion 0.

Überschriften

```
# Kapitel
## Abschnitt
### Unterabschnitt
#### Detail
```

Kapitel
Abschnitt
Unterabschnitt
Detail

Textauszeichnung

```
**fett**, *kursiv*,
~~durchgestrichen~~,
`Quellcode`
```

fett, *kursiv*, ~~durchgestrichen~~, `Quellcode`

Absätze & Trennlinie

```
Erster Absatz.

Zweiter Absatz nach
einer Leerzeile.

---
```

Erster Absatz.

Zweiter Absatz nach einer Leerzeile.

Listen

```
- erstens
- zweitens

  - eingerückt

- drittens

1. eins
2. zwei
```

- erstens
- zweitens
  - eingerückt
- drittens

1. eins
2. zwei

Untergeordnete Listen um **2 Zeichen** einrücken.

Aufgabenliste

```
- [x] erledigt
- [ ] offen
- [ ] noch offen
```

- [x] erledigt
- [ ] offen
- [ ] noch offen

Tabellen

```
<!-- data-type="none"
  data-sortable="false" -->
| Operator | Wirkung      |
|:---------|:-------------|
| AND      | verkleinert  |
| OR       | vergrößert |
| NOT      | schließt aus|
```

Operator	Wirkung
AND	verkleinert
OR	vergrößert
NOT	schließt aus

`:–` links, `:–:` zentriert, `–:` rechts.

`data-type="none"`: keine Grafik; `data-sortable="false"`: feste Reihenfolge.

Inline-Code & Codeblock

```
Operatoren wie `AND`
und `OR` als Code.

```python
print("Hallo")
```
```

Operatoren wie `AND` und `OR` als Code.

```python
print("Hallo")
```

Mathematische Gleichungen

```
Inline: $A \cdot B$

Block:
$$\neg A \lor (B \land C)$$
```

Inline: $A \cdot B$

Block:

$$\neg A \lor (B \land C)$$

`\land` ∧, `\lor` ∨, `\neg` ¬, `\cdot` ·

Links

```
[Beschreibung](https://...)

<https://example.org>
```

Beschreibung
https://example.org

Sprechende Beschreibung erleichtert die Orientierung.

Fußnoten

```
Wichtig[^1] zum Lernen.

[^1]: Quelle: ...
```

Wichtig[1] zum Lernen.

[1] Quelle: …

Bilder

```
![Alt-Text](pfad/bild.png)

![Venn-Diagramm AND/OR](
  https://.../venn.svg)
```

Bild

Alt-Text in eckigen Klammern — wichtig für Barrierefreiheit.

Audio

```
?[Audio](datei.mp3)

?[Hörbeispiel](https://...
  /aufnahme.ogg)
```

MP3, OGG und SoundCloud als Audioplayer einbetten.

`?` vor dem Link kennzeichnet eine Audioressource.

Medien abspielbar

```
!?[Titel](https://www.youtube
.com/watch?v=VIDEO_ID)

!?[Video](datei.mp4)
```

YouTube, Vimeo und Videodateien werden direkt eingebettet.

`!?` kombiniert Bild und Ton zu einem abspielbaren Video.

Eingebettete Ressource

```
??[Katalog](https://kxp.k10plus.de)

??[Spezifikation](
  https://dublincore.org/...)
```

PDF, GeoGebra, 3D-Modelle und Webseiten direkt anzeigen.

`??` bettet die Ressource ein, statt sie nur zu verlinken.

Zitate & Hinweise

```
> Ein einfaches Zitat.

> [!NOTE]     blau
> [!TIP]      grün
> [!IMPORTANT] lila
> [!WARNING] orange
> [!CAUTION] rot
```

> *Ein einfaches Zitat.*

> **NOTE / TIP / IMPORTANT**
> **WARNING / CAUTION**
> Folgezeilen = Inhalt der Box.

CC-BY-SA 4.0 — Martin Lommatzsch, Freiberg 2026

LiaScript – die Erweiterungen

Alles, was Markdown allein nicht kann: Quiz, Animationen, ausführbarer Code, Sprache.

MINT-the-GAP

Animation & Click-Reveal

```
Sofort sichtbar.

{{1}}
Erst nach einem Klick.

{{2}}
Nach dem zweiten Klick.
```

Sofort sichtbar.

—

nach 1. Klick: …

nach 2. Klick: …

Sprecher-Text (TTS)

```
## Folie
        --{{0}}--
Dieser Text wird vorgelesen,
sobald die Folie erscheint.
```

Wird automatisch gesprochen.

`{{0}}` = beim Öffnen, `{{1}}` = nach 1. Klick …

Stimme im Header: `narrator:`

Quiz: Freitext

```
Welcher Operator verkleinert
die Treffermenge?
[[AND]]
[[?]] Beide Begriffe müssen
      vorkommen.
```

Welcher Operator verkleinert die Treffermenge?

[ ]

Hinweis (`[[?]]`, klappbar)

Mehrere gültige Antworten: weitere `[[. . . ]]` anhängen.

Quiz: Single-Choice

```
- [( )] falsch
- [(X)] richtig
- [[?]] Hinweis
********************

Bravo, alles richtig!

********************
```

○ falsch

● **richtig**

Hinweis (klappbar)

Bravo, alles richtig!

Lösungsblock nach `*****` — erscheint erst bei korrekter Antwort.

Quiz: Multiple-Choice

```
- [[X]] vergrößert Treffer
- [[X]] verknüpft Synonyme
- [[ ]] schließt Begriffe aus
- [[X]] ist Vereinigungsmenge
```

☑ vergrößert Treffer

☑ verknüpft Synonyme

■ schließt Begriffe aus

☑ ist Vereinigungsmenge

`[[X]]` = beliebig viele richtig.

Quiz: Kacheln

```
<!-- data-randomize="true" -->
Himmel: [->[rot|(blau)|grün]]

Gras: [->[(grün)|blau|rot]]
```

Antwortkacheln in die passenden Lücken ziehen.

Die richtige Lösung steht in **runden Klammern**; Alternativen trennt `|`.

LiaScript-Standard, kein Import nötig.

JSXGraph

```
```javascript @JSX.Graph
board.create('point', [1, 2],
  {name: 'A'});
```
```

Interaktive Geometrie, Funktionen und Datenvisualisierung.

Der JavaScript-Code erhält automatisch das Board `board`.

ABCjs: Musiknotation

```
```abc @ABCJS.render
X: 1
M: 4/4
L: 1/4
K: C
C D E F | G4 |
```
```

ABC-Notation als Noten darstellen und direkt abspielen.

`@ABCJS.eval` macht den Notentext zusätzlich editierbar.

Algebrite: Terme prüfen

```
Forme äquivalent um:
[[(x - 2) * (x + 2)]]
@Algebrite.check(`x^2 - 4`)

Ableitung: [[-(K/2) * t^(-3/2)]]
@Algebrite.check(`-(K/2) * t^(-3/2)`)

Ergebnis: [[1/3]]
@Algebrite.check2(1/3,0.01)

Schätzung: [[1200]]
@Algebrite.check_margin(1100,1400)
```

Komplexe Terme in Backticks schützen; äquivalente Schreibweisen werden akzeptiert.

Auch TeX-/LaTeX-Eingaben wie `\frac{1}{3}` werden erkannt und vor der Prüfung umgewandelt.

`check2` ergänzt eine Toleranz; `check_margin` prüft einen Wertebereich.

Speech-Recognition-Quiz

```
<!-- data-solution-button="off" -->
Sprich: Hallo.
[[!]]
@SpeechRecognition(de-DE,Hallo)
```

Prüft eine gesprochene Phrase.

`.withFeedback(...)` zeigt bei Abweichung das erkannte Transkript.

Benötigt Chrome, Edge oder Opera.

Häufige Quiz-Kommentare

```
<!-- data-solution-button="5"
     data-hint-button="3"
     data-randomize="true"
     data-show-partial-solution=
  "true" -->
Frage ...
[[Antwort]]
```

In Aufgabensammlung und Wochenaufgaben regelmäßig genutzt.

Lösung nach 5, Hinweis nach 3 Versuchen; mischen, Teilergebnisse anzeigen und Lösung zeitverzögert freigeben.

Vier LiaScript-Templates — jede benötigte Zeile in den Dokumentkopf schreiben

JSXGraph

```
import:
https://raw.githubusercontent.com/liaTemplates/JSXGraph/main/README.md
```

Codeblock mit `@JSX.Graph`.

ABCjs

```
import:
https://raw.githubusercontent.com/liaTemplates/ABCjs/main/README.md
```

ABC-Block mit `@ABCJS.render`.

Algebrite

```
import:
https://raw.githubusercontent.com/liaTemplates/algebrite/master/README.md
```

Nach Eingabe: `@Algebrite.check(...)`.

Speech Recognition

```
import:
https://raw.githubusercontent.com/LiaTemplates/Speech-Recognition-Quiz/refs/
heads/main/README.md
```

Generisches `[[!]]`-Quiz + Makro.

CC-BY-SA 4.0 — Martin Lommatzsch, Freiberg 2026

SchulLia — Unterricht & Interaktion

Makros für Abgabe, Bewertung, Sprachaufgaben und lernförderliche Rückmeldung.

Abgabe & Auswertung

lia-freeze-v2/main/README.md

```
@Abgabe

@Auswertung(F12;Tab;Time)
```

Erzeugt den Abgabelink.

F12: technische Entwicklertoolsignale; Tab: Tab-/Fensterwechsel; Time: Zeit je Folie. Keine Beweise für Täuschung.

Prüfung & Bewertung

lia-freeze-v2/main/README.md

```
@Exam(60min)

3 + 4 = [[7]]

@ADetails(2;Addition,
          Grundrechenarten)
```

Prüfungsdauer: 60 Minuten.

`@ADetails` ordnet dem unmittelbar vorhergehenden Quiz 2 BE zu.

Nach dem Semikolon folgen optionale, kommagetrennte Themen-Tags. `@Auswertung` berechnet damit Gesamtpunktzahl und Themenbereiche.

Ohne Tags: `@ADetails(2)`; Standardwert: 1 BE. Leerzeile vor dem Makro.

Text markieren

lia-marker/main/README.md

```
<div class="markerquiz">
@markred(Nomen)
@markblue(Verb)
@TextmarkerQuiz
</div>

@markedgreen(Beispiel)
```

Nomen Verb

Zielwörter als Quiz in der erwarteten Farbe markieren; im Erklärmodus erscheint die Worterläuterung in der ausgewählten Sprache.

`@mark(...)` akzeptiert jede Farbe; `@markedCOLOR(...)` zeigt eine fertige Markierung.

Kachelfolgen

lia-kachel/main/README.md

```
@Kachelfolge(`[->[(A)|B]]
             [->[(C)|D]]`)

@KachelfolgeN(`[->[(1)|2]]
              [->[(2)|3]]`)
```

Kacheln zu passenden Zielen ziehen.

`@Kachelfolge`: Reihenfolge egal. `@KachelfolgeN`: Ziele schrittweise aufdecken.

Backticks um die gesamte Folge.

Flexible Spalten (DynFlex)

lia-DynFlex/main/README.md

```
<section class="dynFlex">
<div class="flex-child">Inhalt A</div>
<div class="flex-child">Inhalt B</div>
</section>
```

Erzeugt nebeneinanderliegende, per Ziehen skalierbare Bereiche.

Jeder Bereich braucht `class="flex-child"`; auf kleinen Displays stapeln sie sich.

Inhalte nach Ansicht

lia-board-mode/main/README.md

```
<div data-lia-only="slides">
Nur auf Folien
</div>

<div data-lia-only="presentation">
Nur in der Präsentation
</div>

<div data-lia-only="textbook">
Nur im Lehrbuch
</div>
```

`data-lia-only` zeigt einen Block ausschließlich in der angegebenen Ansicht.

Unterstützt werden `slides`, `presentation` und `textbook`.

Das Template erweitert außerdem Folien und Präsentation auf nahezu volle Breite.

Rechtschreibung & Diktat

lia-orthography/main/README.md

```
@orthography(
 `<!-- data-solution-button="2" -->`,
 `Feler im Saz.`,
 `Fehler im Satz.`)

@diktat(Schmetterling)
```

Fehler im Ausgangstext finden und verbessern.

`@diktat` spricht Wort oder Satz vor; die Eingabe wird geprüft.

Reihenfolge: Quiz-Kommentar, fehlerhafter Text, korrekte Lösung.

Bruchteile markieren

lia-Mathe/main/README.md

```
@circleQuiz(3/8)

@rectQuiz(1/3)

@circleQuizC(3/8,
 `<!-- data-hint-button="2" -->`)
```

Drei von acht Kreissektoren auswählen.

Ein Drittel eines Rechtecks markieren.

`...C` übernimmt Quiz-Kommentare.

Zeitgesteuerte Lösung

lia-timer/main/README.md

```
<!-- data-solution-timer="15s"
     data-solution-timer-start=
     "oncheck"
     data-solution-timer-badge=
     "off" -->
7 + 8 = [[15]]
```

Lösung erst nach 15 Sekunden freigeben.

Start: `immediate`, `oncheck` oder `onclick`.

Globale Quizregeln

lia-globalquiz/main/README.md

```
@global(data-solution-button=3
        data-hint-button=2)

@globalforced(
  data-hint-button=4)
```

Vorgaben für alle folgenden Quizze setzen.

`@global`: lokale Werte dürfen abweichen.
`@globalforced`: lokale Werte überschreiben.

Quiz einzeln zurücksetzen

lia-resetter/main/README.md

```
7 + 8 = [[15]]

@resetter
```

Direkt nach dem Quiz einfügen.

Setzt nur dieses Quiz einschließlich Eingabe und Prüfzustand vollständig zurück.

Handschrift & OCR

lia-canvas-ocr/main/README.md

```
Berechne:
$3x + 2 = 11$
[[x = 3]]
@canvas
```

`@canvas` öffnet direkt am Antwortfeld eine Schreibfläche.

Der markierte handschriftliche Bereich wird erkannt und als LaTeX in das Eingabefeld übertragen.

Annotationen

lia-annotation/main/README.md

```
# Tafelbild

Skizze und Rechnung
```

Zeichenleiste auch über JSXGraph-/`lia-coordinate`-Inhalten: DGS-Konstruktionen bearbeiten und Koordinatensysteme skizzieren.

Benötigt beide Imports; OCR ist enthalten.

CC-BY-SA 4.0 — Martin Lommatzsch, Freiberg 2026

SchulLia — Fächer & Visualisierung

Freie Antworten mit KI, Koordinatensysteme, Funktionen und dynamische Geometrie.

LLM-Quiz: freie Antworten

`lia-llm/main/README.md`

A: Vollständige Lösungsalternativen

```
Erkläre die Fotosynthese.

<!-- data-solution-button="off"
     data-llm-textarea="3" -->
[[Antwort]]
```text @LLMQuiz(0.66;assessmentengine=
       quality;operator=erklaeren;solution
       =1;feedback=1;Rechtschreibung=1;
       Satzbau=1;maxthinkingtime=15s;
       maxthinkingtokens=medium,`Erkläre
       die Fotosynthese.`)
Pflanzen bilden mit Licht aus
Wasser und Kohlenstoffdioxid
Glucose. Dabei entsteht Sauerstoff.
<!-- lia-llm:alternative -->
Mit Lichtenergie bauen Pflanzen
aus Wasser und CO2 Traubenzucker
auf und geben Sauerstoff ab.
```
```

B: Einzelne Kriterien

```
Nenne beide Ausgangsstoffe
der Fotosynthese.

<!-- data-solution-button="off" -->
[[Antwort]]
```text @LLMQuiz(0.55;coverage=1;solution
       =1;feedback=1,`Nenne beide
       Ausgangsstoffe der Fotosynthese.`)
<!-- lia-llm:criterion -->
Wasser ist ein Ausgangsstoff.
<!-- lia-llm:criterion -->
CO2 ist ein Ausgangsstoff.
<!-- lia-llm:solution -->
Die Ausgangsstoffe sind Wasser
und Kohlenstoffdioxid.
```
```

A: `alternative` trennt vollständige Musterlösungen: Eine davon muss sinngemäß erfüllt sein.

`quality` prüft gründlicher; `operator=erklaeren` verlangt eine Erklärung.

`Rechtschreibung=1` und `Satzbau=1`: nach der Inhaltsprüfung auf **Sprache prüfen** klicken.

`15s` und `medium` (512 Tokens) begrenzen zusätzliches Denken.

B: Jeder `criterion`-Marker beginnt eine Anforderung. `coverage=1` verlangt beide; `solution` leitet die Musterlösung ein.

`0.66`/`0.55`: Prüfschwelle. `solution=1`: Lösung nach richtiger Antwort. `feedback=1`: Rückmeldung.

A und B getrennt verwenden: Kriterien nicht mit `alternative` oder `operator` mischen.

Jeweils den gesamten Makroaufruf in einer Quelltextzeile schreiben; zweiter Parameter ist die vollständige Frage.

Gleichungslücke

`lia-Mathe/main/README.md`

```
$\frac{\liaquiz}{3}
 = \frac{5}{15}$
@liaQuiz(1)

$x + \liaquiz = 7$
@liaQuizC(3,`<!-- ... -->`)
```

`\liaquiz` setzt ein Eingabefeld direkt in eine Gleichung.

Das folgende Makro liefert die Lösung.

Koordinatensystem

`lia-coordinate/main/README.md`

```
@Koordinatensystem(`xmin=-5;
  xmax=5;ymin=-4;ymax=4;
  width=800;id=B`)

@AchsenBeschriftung(`id=B;
  xlabel=$x$;ylabel=$y$`)
```

System `B` mit Wertebereich und Breite anlegen.

Achsen anschließend beschriften.

Zuerst JSXGraph, danach `lia-coordinate` separat importieren.

DGS: bauen & exportieren

`lia-coordinate/main/README.md`

```
@DGS(`B;tools=[200;310;700];
  restrictions=[200;300]`)

@Zirkel(`B`)

@Geodreieck(`B`)
```

1. Bauen: Im DGS Punkte, Geraden, Funktionen und Konstruktionen anlegen.

2. Exportieren: Objektliste öffnen und den erzeugten Makroblock kopieren.

3. Einfügen: Makroblock im LiveEditor einsetzen und weiterbearbeiten.

Flächen, Winkel & Kreise

`lia-coordinate/main/README.md`

```
@Flaeche(`B;[A;P;C];#e9c46a;
  0.25;inhalt=1;umfang=1`)

@Winkel(`B;alpha;[A;P;C];
  #e76f51;0.8;Wert=1`)

@Kreis(`B;k;M;#264653;0.2;
  radius=P;inhalt=1`)

@Kreissektor(`B;[M;P;Q];#e9c46a;
  0.25;s;inhalt=1;umfang=1`)
```

Polygonfläche, Winkel, Kreis und Kreisbogen aus benannten Punkten erzeugen.

`@Kreissektor` verwendet Mittelpunkt, Radiuspunkt und zweiten Schenkelpunkt; `@Kreissegment` ist ein Alias.

Analysepunkte

`lia-coordinate/main/README.md`

```
@Nullstellen(`B;f;blue;N;wert=1`)

@Extrempunkte(`B;f;green;E;wert=1`)

@Wendepunkte(`B;f;magenta;W;wert=1`)

@Schnittpunkt(`B;f;g;orange;S;wert=1`)
```

Nullstellen, Extrem- und Wendepunkte einer benannten Funktion dynamisch bestimmen.

`@Schnittpunkt` markiert Schnittpunkte zweier Funktionen, Geraden oder Kreise.

Funktionen & Werkzeuge

`lia-coordinate/main/README.md`

```
@PlotFunktion(`B;f;0.5*x^2-2;red`)

@Tangente(`B;f;[2;0];magenta;t;T`)

@Tabelle(`n=3;x;f;P;id=B`)

@Regression(`B`)

@Rekonstruktion(`B;2x-1;0.1`)
```

Funktion zeichnen, Tangente im Berührpunkt anlegen und eine Wertetabelle ergänzen.

`@Regression` öffnet Zeichen- und Ausgleichswerkzeuge; `@Rekonstruktion` prüft den Zielterm mit Toleranz.

Punkte & Geraden

`lia-coordinate/main/README.md`

```
@Punkt(`B;A;2;3;#e63946;1;fix`)

@ErzeugePunkt(`B;P;1;2`,`<!-- -->`)

@Gerade(`B;[A;P];#457b9d;g`)

@Vektor(`B;[A;P];#2a9d8f;v`)
```

Punkte anzeigen oder interaktiv erzeugen lassen.

Geraden und Vektoren referenzieren vorhandene Punkte über ihre Namen.

Geometriequizze

`lia-coordinate/main/README.md`

```
@UmfangQuiz(`B;3;12;0.05`,
 `<!-- data-solution-button="5" -->`)

@FlaecheQuiz(`B;3;6;0.05`,
 `<!-- -->`)

@KonstruktionQuiz(
 `B;3;fest;S4,W90,S3`,
 `<!-- data-solution-button="5" -->`)
```

Prüft ein selbst konstruiertes Polygon mit Eckenzahl, Zielwert und absoluter Toleranz.

`@UmfangQuiz`: Umfang 12; `@FlaecheQuiz`: Fläche 6.

`@KonstruktionQuiz` prüft Seiten und Winkel in fester oder offener Anordnung.

Erklärhinweis & Tooltip

`lia-mathpath/master/README.md`

```
[[?]] @Explain

<div class="notip">
Ohne Glossar-Tooltip
</div>
```

`@Explain` erzeugt einen gestuften Lösungshinweis.

Die Themen stammen aus `@ADetails(...)` beim Quiz; passende Erklärlinks werden in `Explain.md` zugeordnet.

`notip` unterdrückt automatische Glossar-Tooltips im umschlossenen Bereich.

Template-Imports

Basis-URL: `https://raw.githubusercontent.com/MINT-the-GAP/`

Den Pfad oben rechts anfügen und die vollständige URL im LiaScript-Kopf unter `import:` eintragen.

Coordinate: zuerst `https://raw.githubusercontent.com/liaTemplates/JSXGraph/main/README.md` importieren.
`lia-resetter/main/README.md`: direkt importieren; Hinweise zum Einzelreset auf Seite 7.

CC-BY-SA 4.0 — Martin Lommatzsch, Freiberg 2026

Gamification — mit lia-loot

Ressourcen, Funde, Werkzeuge, Tore und Geheimnisse — Grundlagen; Erweiterungen auf Seite 6.

lia-loot

Start: Ressourcen, Highscore, Erfolge

```
@Ressourcen(10, 3, 5)

@Highscore(100, 10, 5,
           15, 2)

@achievements
```

Ressourcen: `10` Gold · `3` Diamanten · `5` Energie.

Highscore: `100` Basispunkte · `10` Abzug je Fehlprüfung · `5` je geöffnetem Hinweis · `15` Freiminuten · danach `2`/Min. (sekundengenau).

Aktiviert zwölf feste Erfolge. Aliasse: `@Achievements`, `@Erfolge`.

Gold, Diamanten & Energie

```
@Schatztruhe(3; toc; menu)

@Diamanttruhe(2; anker)

@Energiekiste(3; mode)
```

Gold: `3` je in `toc` und `menu`.

Diamanten: `2` inline; `anker` bindet den Fund an die Folie.

Energie: `3` in `mode` (dritter Ressourcenwert nötig). Erster Wert = Menge; jedes Ziel erhält sie vollständig.

Schlüssel & Schlösser

```
@Schluessel(blau; menu)

@Schloss(check, blau)

@Schloss(toc, rot; anker)
```

`blau` = Schlüsselfarbe; `menu` = Fundort.

Beim Schloss steht erst das Ziel (check/toc), dann die nötige Farbe. Öffnen verbraucht einen passenden Schlüssel.

`anker` bindet ein sonst globales Schloss an seine Folie.

Gemeinsame Fundoptionen

```
anker; 12s
theme=rot
farbmodus=dunkel
annotationen=aus
unsichtbar
erde; pflanze-zauberstaub
```

`anker` = Folienbindung; `12s` = zwölf Sekunden Wartezeit. Weitere Optionen steuern Theme, Farbmodus, Annotationen und Verbergung.

Direkte Schichten `erde`/`pflanze` laufen von außen nach innen; Suffixe: `-unsichtbar`, `-zauberstaub`.

Lupe & Verbergung

```
@Lupe

@Unsichtbar(Geheimer Text)

@Zauberstaub(Hier funkelt etwas)
```

Lupe einsammeln und in der Leiste aktivieren.

`@Unsichtbar` verdeckt vollständig, `@Zauberstaub` hinterlässt ein schwaches Funkeln.

Einzeiliger Inhalt; Spieleffekt, kein Zugriffsschutz.

Aktionswerkzeuge

```
@Schaufel

@Giesskanne
```

Schaufel öffnet Erde, Gießkanne bringt Pflanzen zum Blühen.

Werkzeug immer vor der ersten Pflichtschicht erreichbar machen. Nur ein Aktionswerkzeug ist aktiv; die Lupe kann parallel laufen.

Erde: Block & Inline

```
@Erdhaufen(unsichtbar)
Vergrabener Inhalt
@EndeErdhaufen

Text @Erdhaufen.inline(Hinweis)
```

Blockform umschließt beliebigen LiaScript-Inhalt; Start und Ende stehen allein auf derselben Folie.

`.inline` gibt einzeiligen Inhalt frei und benötigt kein Endmakro.

Pflanze: Block & Inline

```
@Pflanze(zauberstaub)
Verborgener Inhalt
@EndePflanze

Text @Pflanze.inline(Hinweis)
```

Erst gießen, dann die Blüte anklicken: Danach wird der Inhalt sichtbar.

Aliasse: `@Blume`, `@EndeBlume` und `@Blume.inline`.

Puzzleteile & Puzzletore

```
@Puzzleteil(rot; 1)
@Puzzleteil(rot; 2)
@Puzzletor(rot; [[2;1]])

@Puzzleteil(blau; 1)
@Puzzletor(blau; [[1]]; anker)
```

`1/2` = Teilnummern; `[[2;1]]` = Teil 2 links, Teil 1 rechts. Alle Teile stehen davor; maximal 16.

Ohne `anker`: Navigationstor. Mit `anker`: Inhalt bis zur nächsten `#`-Überschrift.

Zweiweg- & Einwegportale

```
@Portal(5)
@Portal(5; einweg)

@Einwegportal(5)
@Einbahnportal(5)
```

`5` = Ziel ist die fünfte Kursfolie (1-basiert). `@Portal` ist standardmäßig ein Zweiwegportal mit Rückweg.

Die letzten drei Formen sind einweg; `@Einbahnportal` ist ein Alias.

Bedingte Bereiche

```
@lootif(Gold >= 5; spawn)
Dieser Inhalt erscheint dauerhaft.
@Endelootif
```

`5` = Schwelle: Der Bereich erscheint ab fünf Gold dauerhaft. Einzig gültige Aktion: `spawn`.

End-Aliasse: `@EndeLootif`, `@endlootif`, `@EndLootIf`.

Start und Ende allein, foliengleich und korrekt verschachtelt.

Geheimfolie

```
## Das geheime Labor

@Geheimfolie
```

Verbirgt die Folie vor ToC und normaler Navigation. Erreichbar per exakter Titelsuche oder gezieltem Portal.

Kein Schutz für vertrauliche Inhalte: Quelle und Seitennummer bleiben prinzipiell einsehbar.

Import: `https://raw.githubusercontent.com/MINT-the-GAP/lia-loot/main/README.md`
Nur öffentliche Makros verwenden — niemals interne `@Loot..._`-Aufrufe.

Kosten: Hinweis 1 Gold · Auflösen 1 Diamant · Prüfen 1 Energie. **Ziele:** `toc`, `menu`, `classroom`, `info`, `translator`, `mode`.
Schloss zusätzlich: `seitenwechsel`, `check`, `resolve`, `hint`, `portal`. **Trigger:** Aufgaben, Ressourcen/Kisten, Schloss/Puzzle, Geheimfolie/Lupe/Marker.

CC-BY-SA 4.0 — Martin Lommatzsch, Freiberg 2026

Gamification — weitere Möglichkeiten

Nebel, Werkzeuge, Shop, Begleiter und Energiereserve — Erweiterungen zu Seite 5.

lia-loot

Taschenlampe & Nebel

```
@Taschenlampe

@Nebel(Verborgener Hinweis)

@Schatztruhe(nebel)
```

Taschenlampe einsammeln und in der Leiste aktivieren. Im Lichtkegel werden vernebelte Inhalte les- und bedienbar.

`nebel` ist eine Fundoption; `farbmodus=dunkel` prüft dagegen den gewählten Farbmodus.

Ganze Bereiche vernebeln

```
@NebelStart
Dieser Absatz ist verdeckt.

@Diamanttruhe
@NebelEnd
```

Umschließt auch Listen, Bilder, Quizze und Makros. Start und Ende stehen allein auf derselben Folie.

Die Taschenlampe muss vorher erreichbar sein. Nebel ist mit Erde, Pflanzen und Lupenverstecken kombinierbar.

Axt & Holzkisten

```
@Axt

@Kiste(@Schatztruhe)

@Perk(eisenaxt)
@Perk(goldaxt)
@Perk(diamantaxt)
```

Axt einsammeln, aktivieren und die Kiste aufbrechen. Sie enthält genau ein Item.

Hiebe je Kiste: Stein 8, Eisen 4, Gold 2, Diamant 1. Es gilt die beste Stufe; die Axt bleibt erhalten.

Geschenke & verschachtelte Items

```
@Geschenk(@Katze)

@Geschenk(`@Schluessel(blau)`)

@Kiste(`@Katze(grau)`)
```

Ein Geschenk öffnet sich per Klick; sein Inhalt entsteht erst danach.

Hat das innere Item eigene Klammerparameter, seinen **vollständigen Aufruf in Backticks** setzen. Gilt auch für Kiste und Futter.

Shop: Angebote & Preise

```
@Shop(schaufel=3gold; energie-5=2
      gold)

@Shop(lupe=2diamanten; giesskanne=3
      gold+1diamant)
```

`artikel=preis`; Angebote mit Semikolon trennen. Ressourcenartikel tragen die Menge im Namen.

Benötigt `@Ressourcen`; Energieangebote zusätzlich dessen dritten Wert. Jedes konkrete Angebot ist einmal kaufbar.

Atlaskarte: Fortschritt sichtbar machen

```
@Atlaskarte

@Perk(atlas-kurszahlen)

@Perk(atlas-folieninfo)
```

Die Karte öffnet die Übersicht der zwölf festen Erfolge.

`atlas-kurszahlen` ergänzt genaue Bestände und Fortschritt; `atlas-folieninfo` die noch offenen Funde auf der aktuellen Folie.

Perks: dauerhafte Verbesserungen

```
@Perk(energiekistenbonus-10)

@Perk(lupenradius-15)

@Perk(taschenlampenradius-20)
```

Mehr Energie aus Energiekisten oder größerer Suchradius. Zahl = Prozentwert (1–500); Prozent-Perks addieren sich.

Nichtganzzahlige Zusatzenergie wird ausgewürfelt. Dieselben Artikel sind auch im Shop erhältlich.

Katze, Futter & Halsbänder

```
@Katze(grau)

@Futter(@Energiekiste)

@Perk(katzenhalsband-blau)
```

Begleiter reagiert auf Funde und richtige Antworten. Fell: orange, grau, schwarz, weiß oder dreifarbig.

Futter einsammeln, aktivieren und Katze anklicken: Das enthaltene Item wird automatisch eingesammelt. Halsbänder wechseln die Farbe.

Reservefolie: neue Energie verdienen

```
## Energiereserve

@Reservefolie(2)

Berechne die Summe.

3 + 3 = [[6]]
```

Genau eine Reservefolie je Kurs; Energie in `@Ressourcen` aktivieren. Beim Übergang von positivem Bestand auf 0 öffnet sie automatisch.

Prüfen ist hier kostenlos. Jedes erstmals korrekt gelöste Quiz gibt 2 Energie; danach automatisch zurück. Weitere Reservequizze ermöglichen spätere Notfälle.

Ressourcenbonus & Timerkosten

```
@Ressourcen(10, 3, 5, goldwert=100,
      diamantwert=250)

@Ressourcen(10, 3)
```

Am Kursende geben übriges Gold und Diamanten zusätzliche Highscorepunkte; Standard: 100 bzw. 250 je Einheit. Energie gibt keinen Bonus.

Ohne dritten Zahlenwert kein Energielimit. Mit Energie kostet auch ein erfolgreicher manueller Timerstart (`onclick`) 1 Energie.

Funde & Schlösser in anderen Templates

```
@Schatztruhe(canvasocr)

@Schloss(dynflex, blau)

@Diamanttruhe(coordinate; anker)
```

Das Zieltemplate **direkt im Kurskopf importieren**; bei lokalen Zielen dessen Komponente auf derselben Folie anlegen.

Weitere Ziele: `timer`, `boardmode`, `marker`, `markerquiz`, `annotation`, `kachel`, `llm`, `freeze`, `mathpath`.

Import: `https://raw.githubusercontent.com/MINT-the-GAP/lia-loot/main/README.md`

Spielpfad planen: Werkzeuge vor ihren Verstecken erreichbar machen; Shoppreise und Pflichtkosten mit Fehlerreserve abdecken. Die Reservefolie löst keinen Sprung aus, wenn der Kurs bereits mit 0 Energie startet.

Stand: Oktober 2026

CC-BY-SA 4.0 — Martin Lommatzsch, Freiberg 2026

Weitere Bausteine

Rechenwege, Einheiten, Geometrie und Quizsteuerung — Ergänzungen zu den Seiten 2 bis 4.

MINT-the-GAP

OCR: vollständige Rechenwege

```
Löse die Gleichung.

@BerechneOCR(`3*x+2=11`,1)
[[?]] Ziehe zuerst 2 ab.

@BerechneOCR(`347+185`)
```

`lia-canvas-ocr`: erzeugt Quiz, Zeichenfläche und Rechenwegprüfung. Zweites Argument `1`/`0`: Zeilenrückmeldung an/aus.

Reine ganzzahlige Grundrechenaufgaben wählen automatisch das passende schriftliche Verfahren. Algebrite direkt mitimportieren.

OCR: Ableitung, Integral & mehr

```
@BerechneOCR(`f(x)=x^3`,`aufgabe=
        ableitung;ordnung=1;
        zeilenrueckmeldung=1`)

@BerechneOCR(`f(x)=x^2`,`aufgabe=
        integral;von=0;bis=2`)
```

Benannte Optionen gemeinsam in **ein** Backtick-Argument schreiben.

Weitere Aufgaben: Nullstellen, Extrempunkte, Tangente und Kurvendiskussion. Je nach Typ `stelle`, `intervall` oder `teile` ergänzen; mathematische Abdeckung in der README prüfen.

Algebrite: Einheiten mitprüfen

```
Länge: [[1.5km]]
@Algebrite.check(1.5km,units=1)

Länge: [[1.5km]]
@Algebrite.check2(1.5km,20m)
```

`1500m` und `1.5km` sind gleichwertig. Fehlende oder falsche Einheit wird abgelehnt.

`check` und `check_expression`: Einheiten per `units=1` aktivieren. Bei `check2` und `check_margin` bereits an; `units=0` schaltet sie aus.

Algebrite: Gleichungen & mehrere Felder

```
[[x^2-1=2x]]
@Algebrite.check_expression(`x^2-2*x
        -1=0`)

$a=$ [[1/3]], $b=$ [[2/3]]
@Algebrite.check2([1/3;2/3],0.01)
```

Gleichungsprüfung vergleicht links minus rechts; kein allgemeiner Vergleich von Lösungsmengen.

Mehrere Sollwerte als Semikolonliste in Feldreihenfolge. Eine Toleranz gilt für alle; alternativ eine Liste je Feld angeben.

Geometrie: mehrere Bedingungen zugleich

```
@GeometrieQuiz(`B;4;Form(Rechteck;
        exklusiv=Quadrat);Flaeche
        (20;0.05);Umfang(18;0.05)`,`
        <!-- -->`)
```

`lia-coordinate`: auf einem interaktiven Board `B` mit DGS. Alle Bedingungen gelten für **dasselbe** selbst konstruierte Polygon.

Hier: Rechteck, kein Quadrat, Fläche 20 und Umfang 18. Weitere Bedingung: `Konstruktion(...)`. Alias: `@KoordQuiz`.

Koordinaten als statisches SVG

```
@Koordinatensystem(`xmin=-3;xmax=3;
        ymin=-2;ymax=4;id=S;static
        =1`)

@Punkt(`S;A;1;2;red;1;fix`)
```

`static=1` bzw. `statisch=1` rendert unterstützte feste Objekte als skalierbares SVG.

Für Abbildungen; keine DGS-Interaktion und keine interaktiven Quizze. `border=0` allein aktiviert diesen Modus nicht.

Funktionsscharen & Regler

```
@Schar(`f;x;mx+n;B;term=1;blue`)

@Regler(`B;a;-3;3;0.1;1;red`)
@PlotFunktion(`B;g;a*x^2;red`)
```

Zuerst ein interaktives `lia-coordinate`-Board `B` anlegen (Seite 4).

`@Schar` erzeugt die Parameterregler zur Funktionsschar. Ein eigener `@Regler` steuert hier a von −3 bis 3 in 0,1-Schritten, Startwert 1.

Polyominos: eine Zielsumme legen

```
@PentominoQuiz(45,`name=P;type=I2;
        numbers=[1,2]`,`<!-- data-
        solution-button="off" -->`)

@PentominoDockQuizAuswahl(65,`I2,L3,
        O4`,`<!-- data-solution-
        button="off" -->`)
```

`lia-pentominos`: einen Stein verschieben und drehen oder aus dem Dock wählen.

Ein einzelner Stein muss die Zielsumme erreichen; Steinsummen werden nicht addiert. Suffix `N`: Feld von 50 bis −49. Importfolge: JSXGraph, Coordinate, Pentominos.

Hinweise zeitgesteuert freigeben

```
<!-- data-hint-timer="20s"
 data-hint-timer-start="oncheck"
 data-hint-badge="off" -->
7 + 8 = [[15]]
[[?]] Ergänze zuerst auf 10.
```

`lia-timer`: Hinweis nach 20 Sekunden ab dem ersten Prüfen. Unabhängig vom Lösungstimer.

Start auch `immediate` oder `onclick`. Badge heißt hier genau `data-hint-badge`; Starttext über `data-hint-timer-start-label`.

Freeze: Bewertung erst bei Abgabe

```
@Abgabe

@Auswertung(F12;Tab;Time;Send)
```

`Send` hält Prüfen während der Bearbeitung neutral; Korrektheitsfeedback und Lösung bleiben verborgen. Bewertet wird beim Erstellen des Freeze-Links.

`F12`: technische Entwicklertoolsignale; `Tab`: Fokuswechsel; `Time`: Minuten je Folie. Die Signale belegen keine Täuschung.

Native Auswahl & Matrixquiz

```
Der Himmel ist
[[rot|(blau)|grün]].

Ordne die Zahlen zu.

[[ gerade ] [ ungerade ]]
[   (X)         ( )    ] 2
[   ( )         (X)    ] 3
```

Kein Import nötig. Bei der Auswahl steht die richtige Antwort in runden Klammern.

Matrix: gemeinsame Spaltenüberschriften, eine Auswahl je Zeile. Eckige Marker `[X]` statt `(X)` erlauben mehrere richtige Antworten je Zeile.

Textkorrektur & Strichlisten

```
@orthographytext(`<!-- data-solution
        -button="2" -->`,`Der Hnd
        rennt.`,`Der Hund rennt.`)

@Strichliste(12)
```

`lia-orthography`: mehrzeiliges Eingabefeld zur Textkorrektur, Argumentfolge wie bei `@orthography`.

`lia-Mathe`: Strichliste mit Fünferbündelung; auch inline oder in Tabellen verwendbar.

Direkte Imports

Für jedes genannte SchulLia-Template im Kopf:
`import: https://raw.githubusercontent.com/MINT-the-GAP/<Repo>/main/README.md`
<Repo> ersetzen; URLs und Makroaufrufe ohne Druckumbrüche übernehmen. Algebrite/JSXGraph: Import-URLs auf Seite 2.

Einzelreset

`@resetter` nach dem Quiz. Pro Abschnitt nur ein zurücksetzbares Coordinate-Quiz je Board-ID. Kachelquizze und andere Quiztypen für verlässlichen Einzelreset im unveränderten LiaScript-Core auf getrennte Abschnitte legen.

Navigation & globale Regeln

`lia-navigation` direkt importieren: aufklappbares Inhaltsverzeichnis mit Lesezeichen; kein zusätzlicher Makroaufruf. `data-randomize`, `data-score` und `data-max-trials` weiterhin lokal am Quiz setzen.

CC-BY-SA 4.0 — Martin Lommatzsch, Freiberg 2026